Activity 2: Jenkins CI/CD Pipeline with Terraform Infrastructure Provisioning

Nov 9, 2026 · @Harshada

1. Aim and objectives

Student: Harshada Gahininath Gujar · **GitHub:** Harshadagujar2005

This activity built a working CI/CD pipeline in Jenkins that pulls code from GitHub, builds and tests it, then validates, plans and applies a Terraform configuration that creates an AWS EC2 instance. The pipeline ran end to end and finished with `Finished: SUCCESS`, and the EC2 instance `jenkins-terraform-demo` was created in the Asia Pacific (Mumbai) region.

Objectives:

- Configure a Jenkins pipeline job that retrieves source code from a GitHub repository.
- Write a `Jenkinsfile` with stages for checkout, build, test and Terraform.
- Write Terraform files that define an EC2 instance and its security group.
- Validate the configuration with `terraform init`, `validate` and `plan`, then provision it with `apply`.
- Demonstrate the pipeline and explain how CI/CD and Infrastructure as Code (IaC) work together.

2. Tools and environment

Everything ran on a Windows laptop inside WSL2 (Ubuntu 26.04), with Jenkins and Terraform on the same Linux machine and the EC2 instance created in the AWS Mumbai region.

Tool	Version	Purpose
WSL2 with Ubuntu	26.04	Linux environment on Windows
OpenJDK	21.0.12.1	Runtime required by Jenkins
Git	2.53.0	Source control and checkout
Node.js / npm	v20.20.2 / 10.8.2	Build and test of the sample app
Jenkins	LTS, from the official apt repository, at `http://localhost:8080`	CI/CD server
Jenkins plugins	Git, Pipeline, Credentials Binding	Checkout from GitHub, pipeline stages, secure AWS credentials
Terraform	v1.17.1 (linux_amd64)	Infrastructure as Code
Terraform AWS provider	hashicorp/aws 5.100.0	Talks to the AWS API

Tool	Version	Purpose
AWS	EC2 in ap-south-1 (Mumbai)	Cloud infrastructure
GitHub	Public repository	Source code hosting

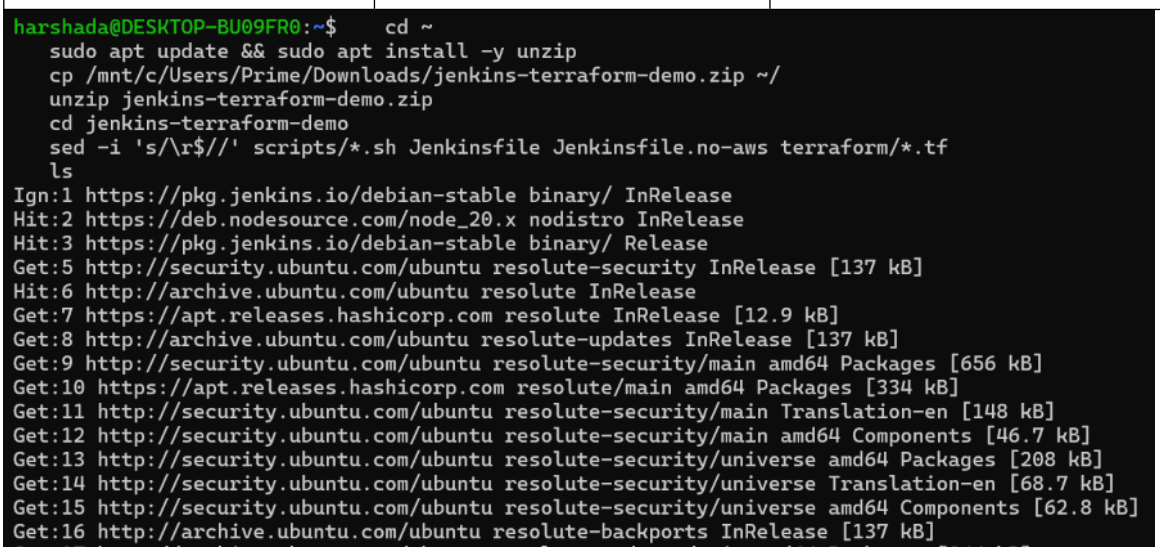


Figure 1: Project folder copied into the Ubuntu (WSL2) home directory.

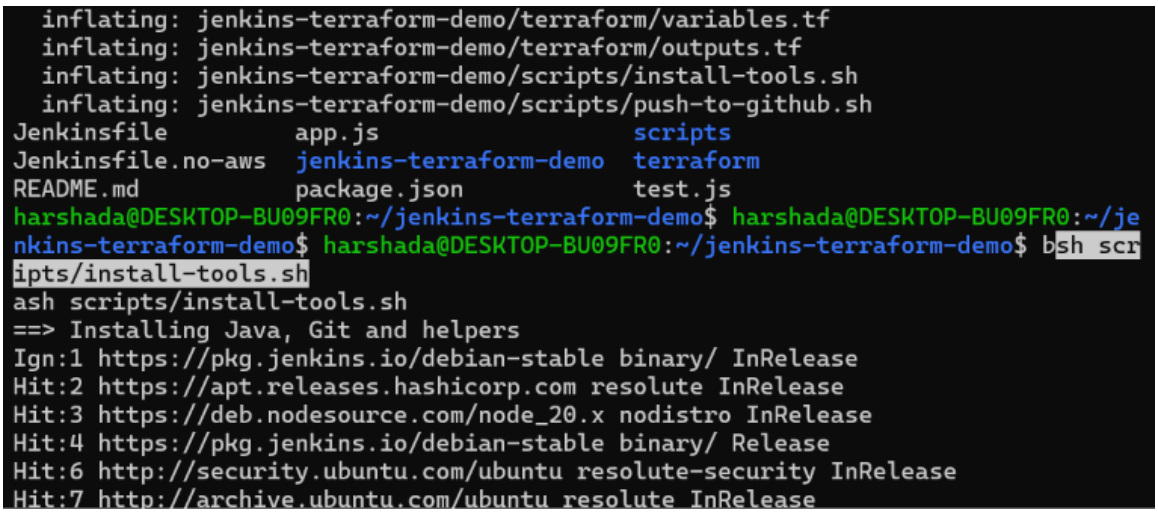


Figure 2: Install script started, setting up Java, Node.js, Jenkins and Terraform.

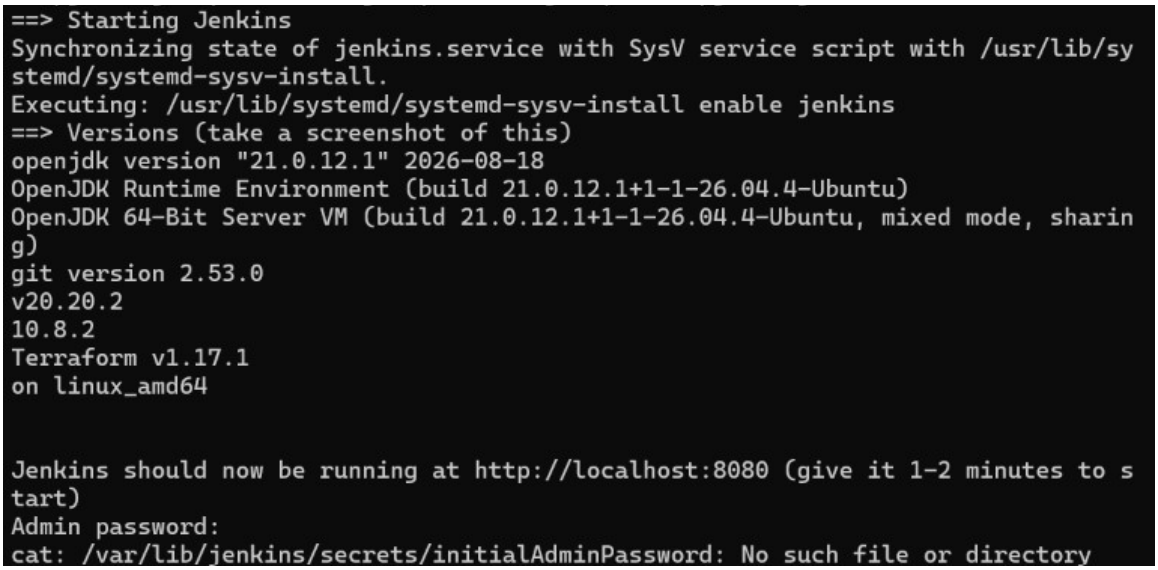


Figure 3: Installed versions of Java 21, Git, Node.js, npm and Terraform.

3. Source code repository

The whole project is in one public GitHub repository: [Harshadagujar2005/jenkins-terraform-demo] (branch `main`). It holds a small Node.js application, its tests, the `Jenkinsfile` and the Terraform configuration.

Path	What it does
`app.js`	Sample application with `add` and `greet` functions
`test.js`	Unit tests using Node's built-in test runner
`package.json`	Defines `npm run build` (`node --check app.js`) and `npm test` (`node --test`)
`Jenkinsfile`	Declarative pipeline: checkout, build, test, Terraform init, validate, plan, approval, apply
`Jenkinsfile.no-aws`	Reduced pipeline without AWS (init and validate only), kept as a fallback
`terraform/main.tf`	Provider, AMI lookup, security group and EC2 instance
`terraform/variables.tf`	Region and instance type variables
`terraform/outputs.tf`	Outputs the instance ID and public IP
`scripts/`	Helper scripts to install the tools and push to GitHub
`docs/` and `logs/`	This report, screenshots and execution logs

4. Jenkins configuration and GitHub integration

Jenkins is running on the local Linux machine, is connected to the GitHub repository, and checked out the code successfully on its first build (Figures 4 to 9).

1. **Install and start Jenkins.** Jenkins was installed from its official apt repository and started as a service. It needs Java 21, so OpenJDK 21 was installed first. The Jenkins dashboard opened at `http://localhost:8080` (Figure 4).
2. **Check plugins.** The Git, Pipeline and Credentials Binding plugins are installed and enabled.
3. **Store the AWS key securely.** A credential of kind *Username with password*, scope Global, ID `aws-creds`, holds the AWS access key for Terraform (Figures 5 and 6). The secret is masked in Jenkins and in build logs, and is never committed to GitHub. The pipeline reads it only inside a `withCredentials` block.
4. **Create the pipeline job.** A new item of type *Pipeline* named `jenkins-terraform-pipeline` was created (Figure 7).
5. **Connect it to GitHub.** The job uses *Pipeline script from SCM* with SCM set to Git, the repository URL `https://github.com/Harshadagujar2005/jenkins-terraform-demo.git`, branch `*/main`, script path `Jenkinsfile` and no credentials, because the repository is public (Figure 8).

The first build log confirms the integration: Jenkins reports `Obtained Jenkinsfile from git https://github.com/Harshadagujar2005/jenkins-terraform-demo.git`, then clones the repository into its workspace (Figure 9).

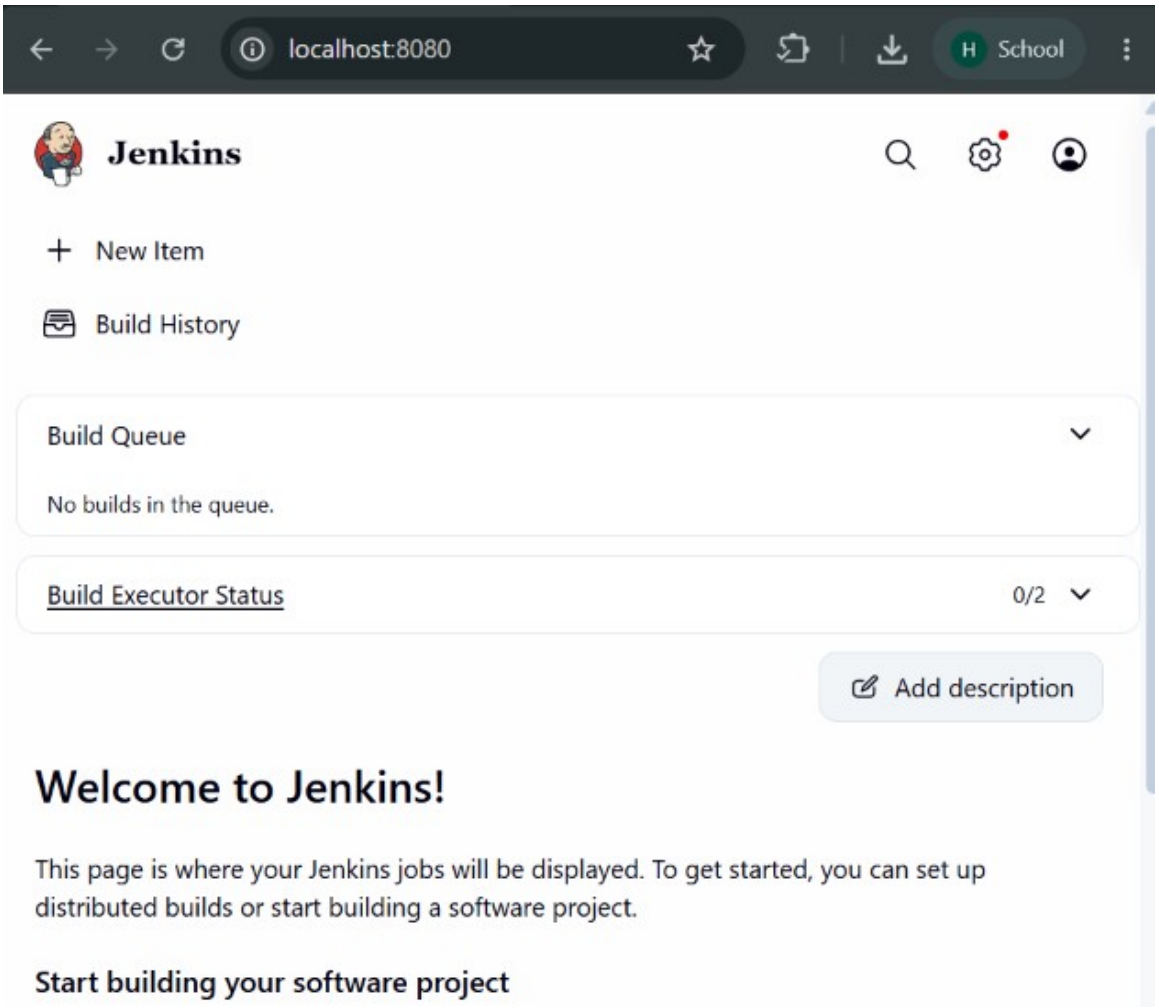


Figure 4: Jenkins dashboard running at localhost:8080.

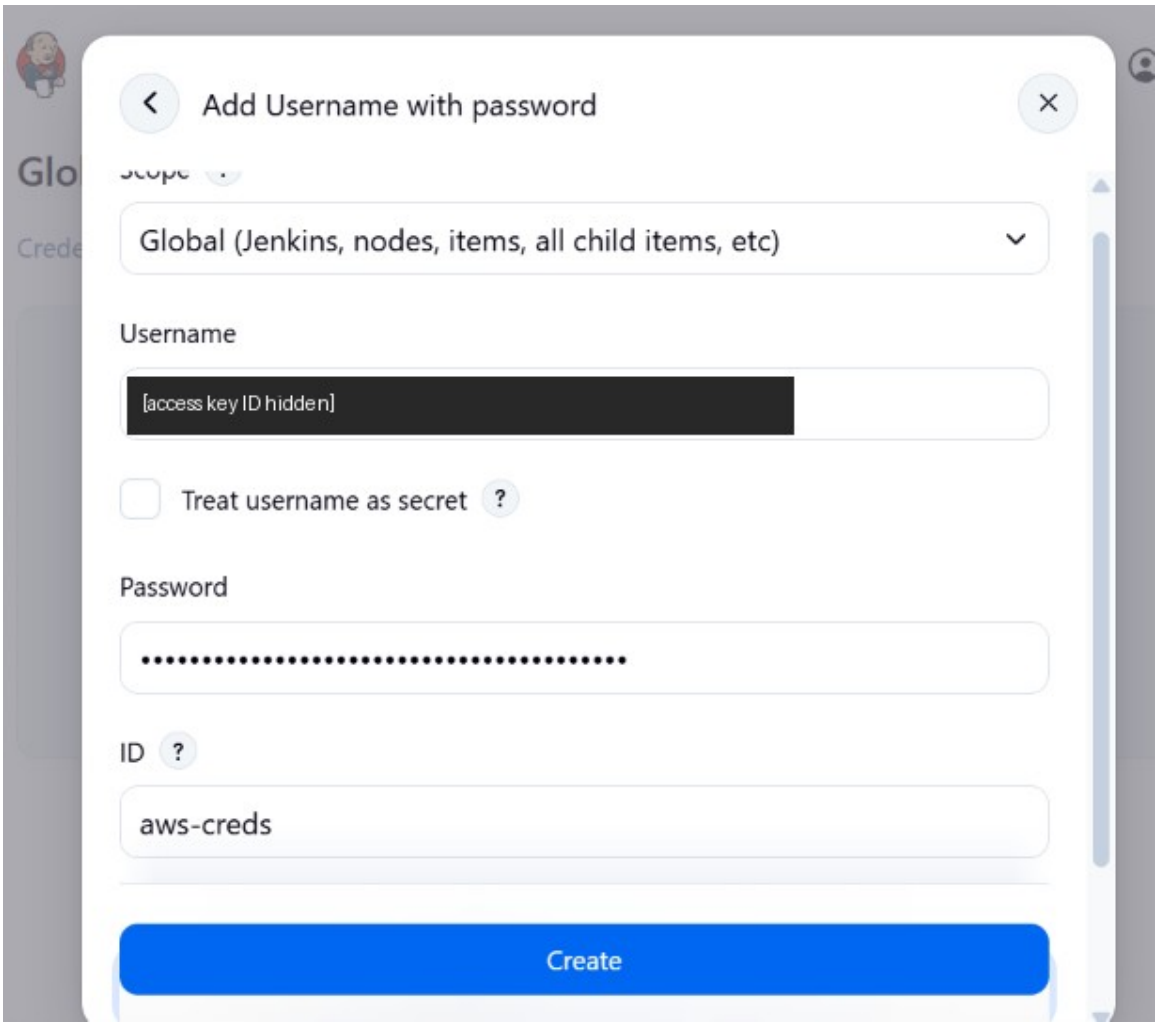


Figure 5: Adding the `aws-creds` credential. The key ID is hidden here and the secret is masked.

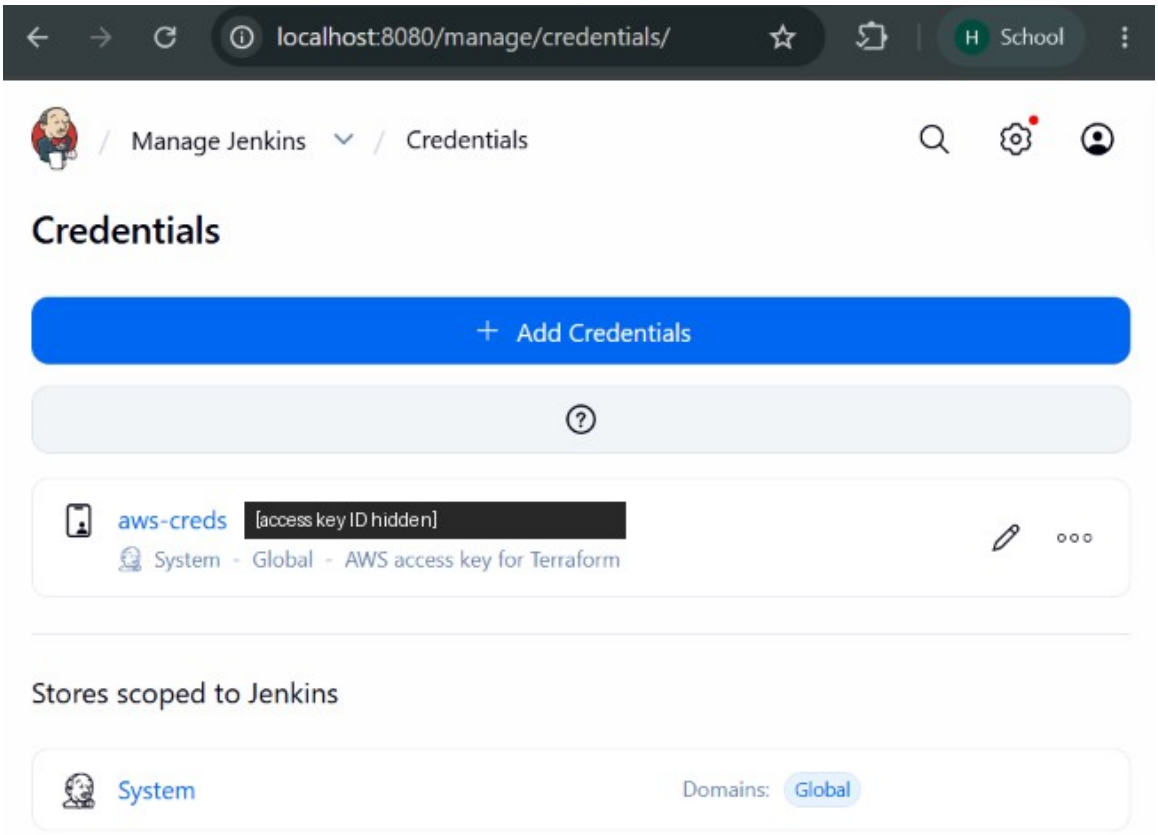


Figure 6: The `aws-creds` credential listed in Jenkins (key ID hidden).

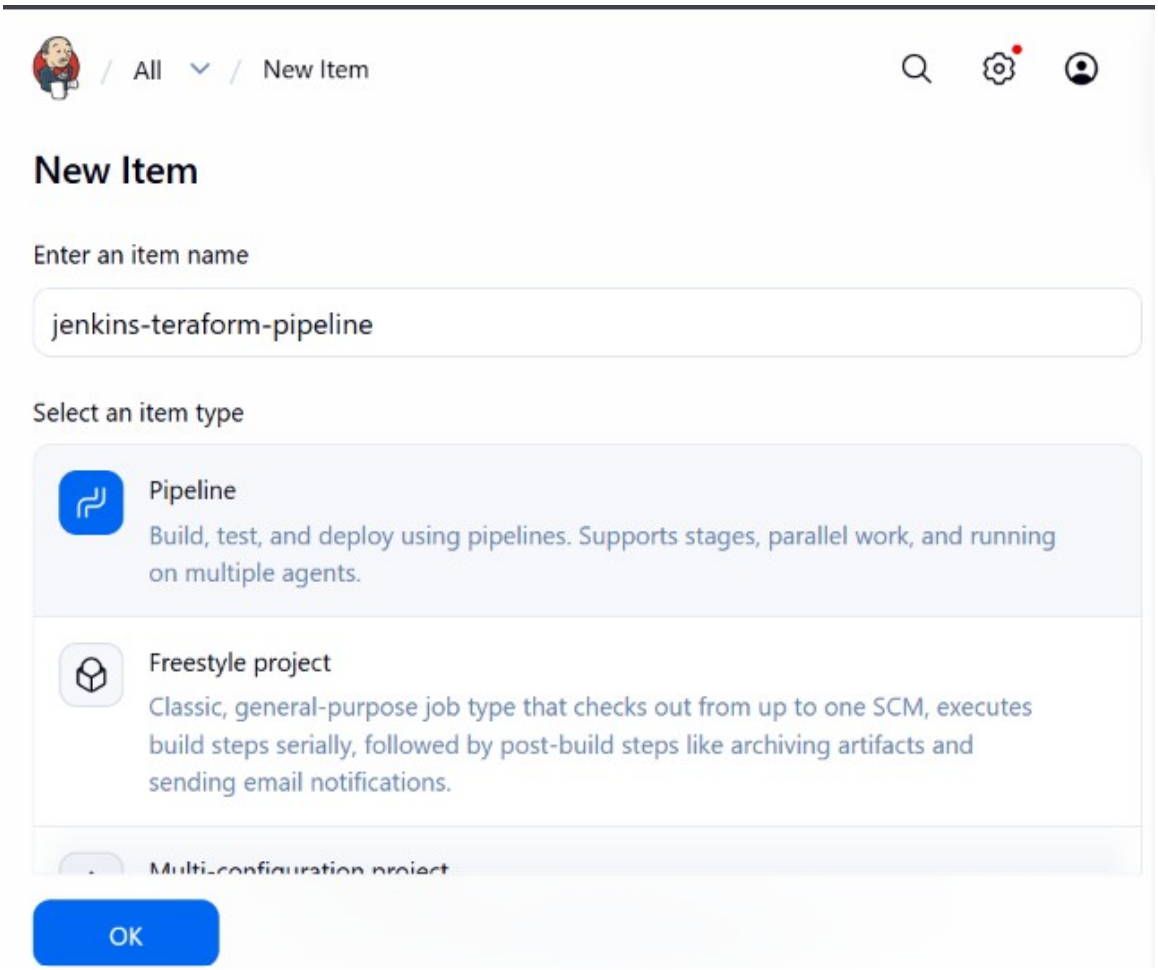


Figure 7: Creating a new Pipeline item for the job.

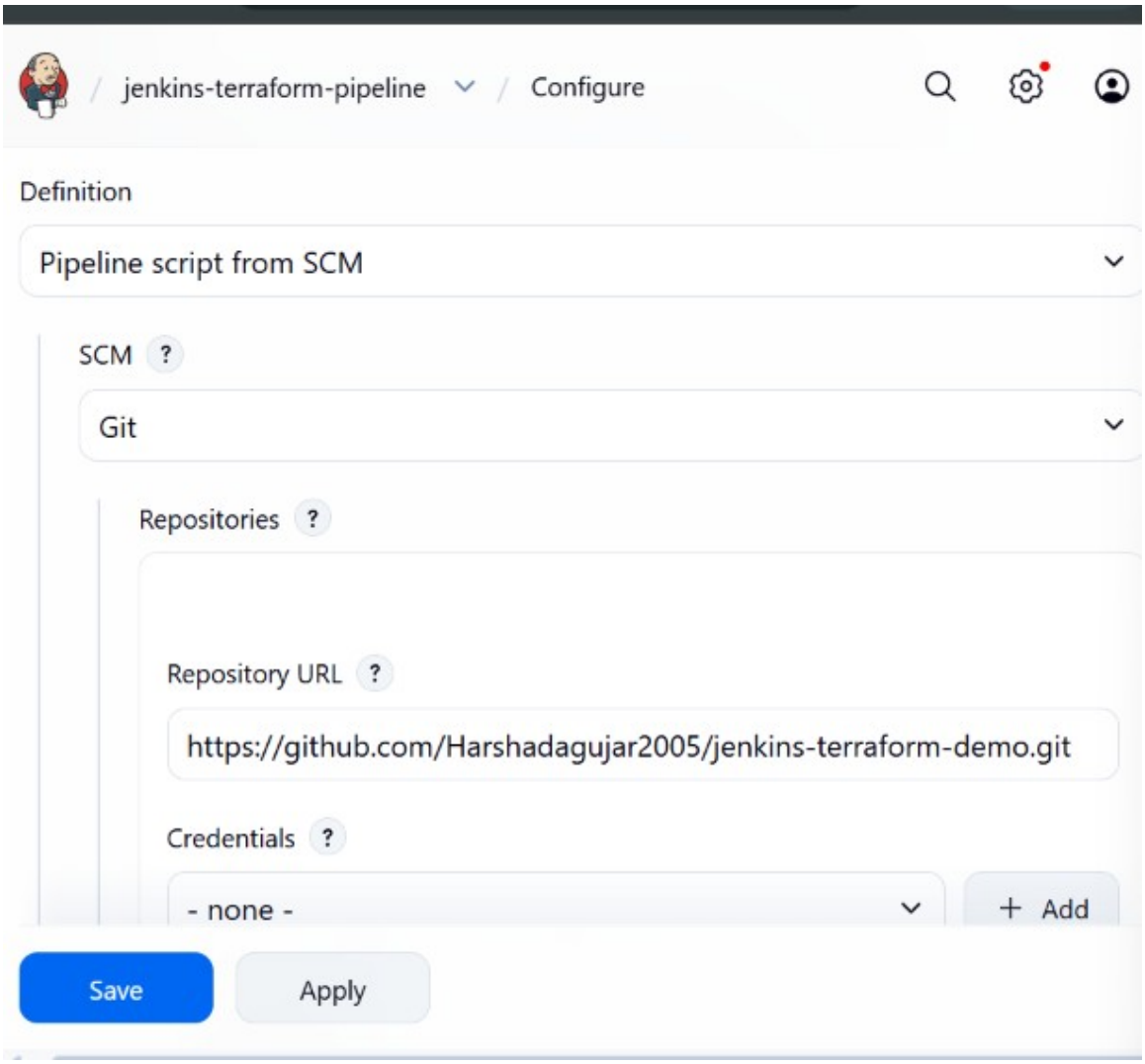


Figure 8: Pipeline script from SCM, with Git and the GitHub repository URL.

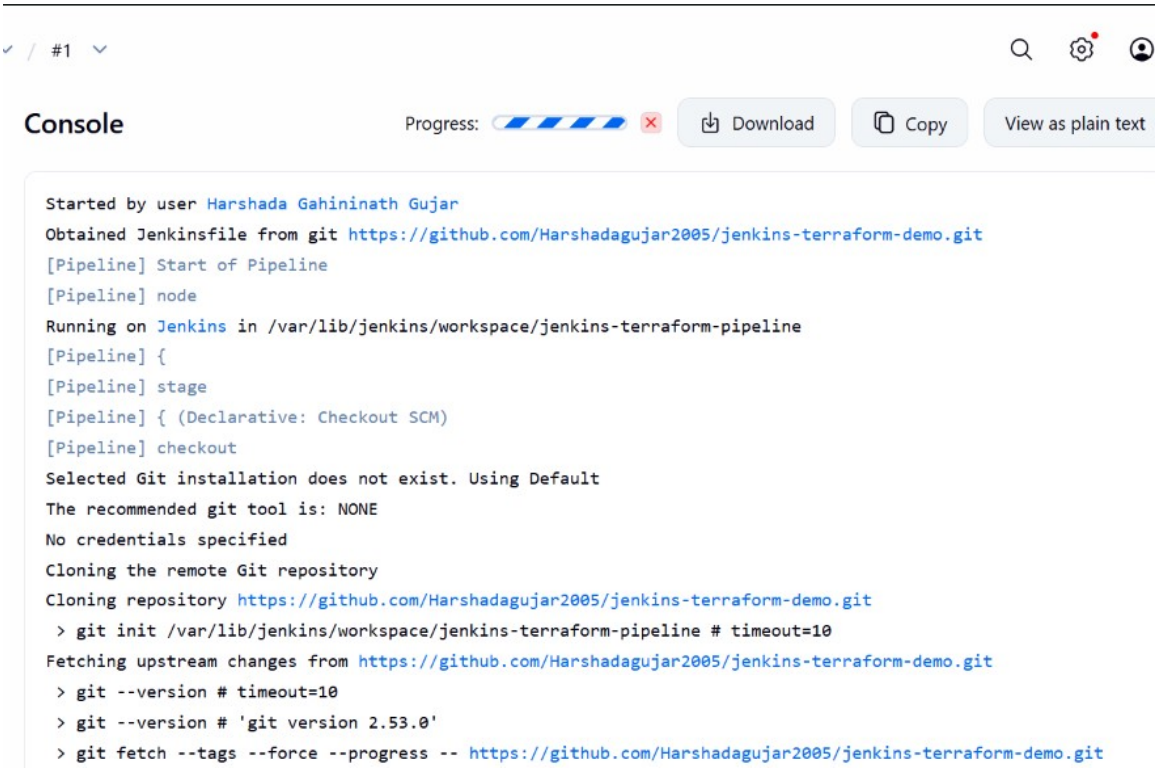


Figure 9: Console output of build #1 obtaining the Jenkinsfile from GitHub and cloning the repository.

5. Jenkinsfile and CI/CD pipeline

The `Jenkinsfile` defines a declarative pipeline with eight stages, and build #1 ran through all of them and ended with `Finished: SUCCESS` (Figures 10 to 14).

Stage	What it does	Command
Checkout	Gets the source from GitHub	`checkout scm`
Build	Checks the Node version and the app's syntax	`npm run build`
Test	Runs the unit tests	`npm test`
Terraform Init	Downloads the AWS provider and prepares the working directory	`terraform init -input=false`
Terraform Validate	Checks formatting and syntax of the configuration	`terraform fmt -check`, `terraform validate`
Terraform Plan	Previews the changes and saves them as `tfplan`, using the `aws-creds` credential	`terraform plan -input=false -out=tfplan`
Approval	Pauses until a person approves creating the	`input` step

Stage	What it does	Command
	instance	
Terraform Apply	Creates the resources from the saved plan	`terraform apply -input=false tfplan`

The Approval stage is a deliberate safety gate. The pipeline stops after the plan, and only a person clicking **Proceed** lets it create real cloud resources. In the demonstration the build was approved by Harshada Gahininath Gujar (Figure 12). A `post` block prints `Pipeline completed successfully` or `Pipeline failed, check the console output`.

Results:

- Local build and test passed before pushing: `npm run build` succeeded and `npm test` reported 6 passed, 0 failed. The three tests in `test.js` ran twice because the local folder held a duplicate copy of the project (Figure 11).
- Build #1 started by the user, checked out the code from GitHub, passed every stage up to the plan, and was approved at the Approval stage.
- After approval, Terraform Apply finished, printed the `public_ip` output, and the build ended with `Pipeline completed successfully` and `Finished: SUCCESS` (Figure 13).

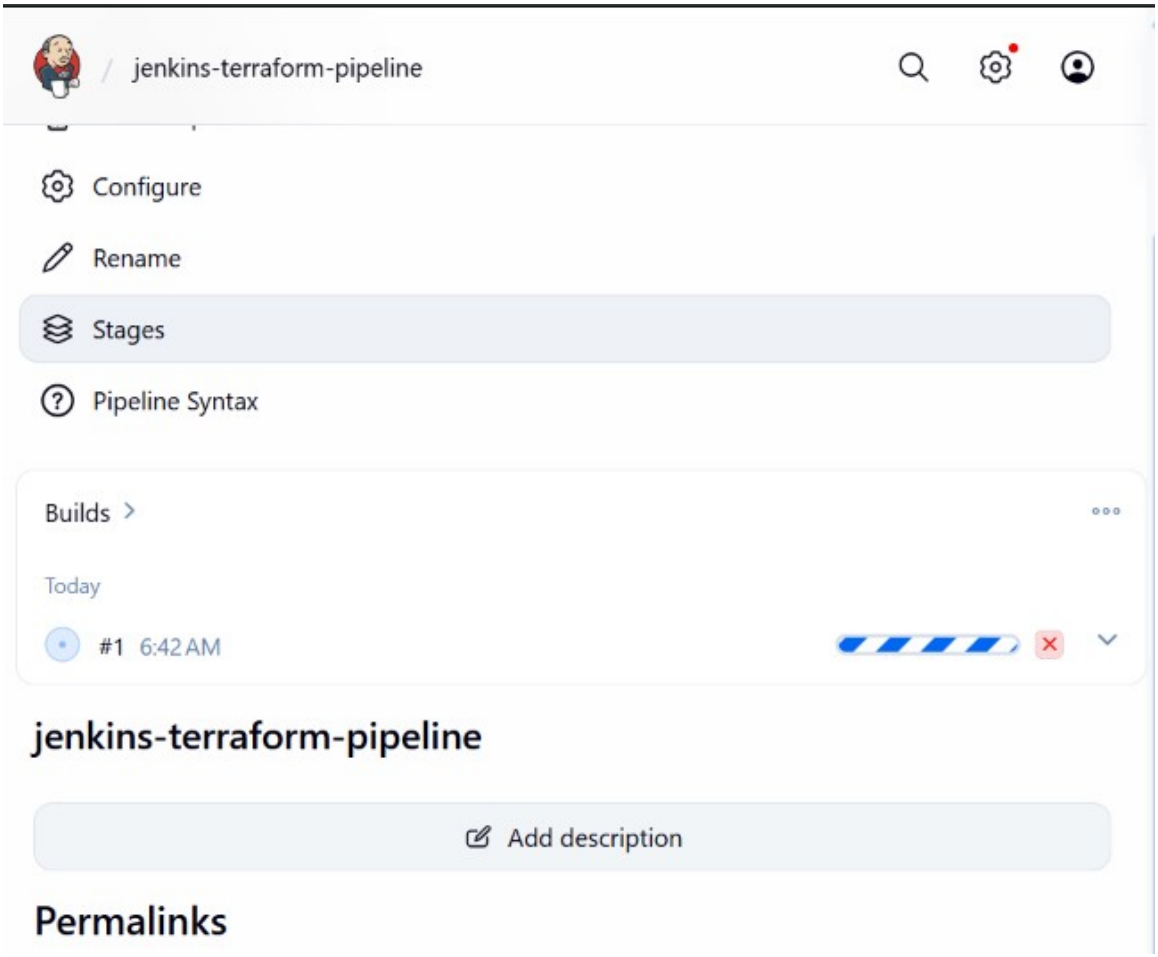


Figure 10: Build #1 of the pipeline running.

```
harshada@DESKTOP-BU09FR0:~/jenkins-terraform-demo$ cd ~/jenkins-terraform-demo
npm run build
npm test

> jenkins-terraform-demo@1.0.0 build
> node --check app.js


> jenkins-terraform-demo@1.0.0 test
> node --test

✔add works (1.919199ms)
✔add handles negatives (1.2017ms)
✔greet works (0.4748ms)
✔add works (2.406799ms)
✔add handles negatives (0.2727ms)
✔greet works (0.3475ms)
ℹtests 6
ℹsuites 0
ℹpass 6
ℹfail 0
ℹcancelled 0
ℹskipped 0
ℹtodo 0
ℹduration_ms 98.96283
```

Figure 11: `npm run build` and `npm test` passing locally before the push.

```
Apply the Terraform plan and create the EC2 instance?
Proceed or Abort
Approved by Harshada Gahininath Gujar
```

Figure 12: Approval stage, approved by the user.

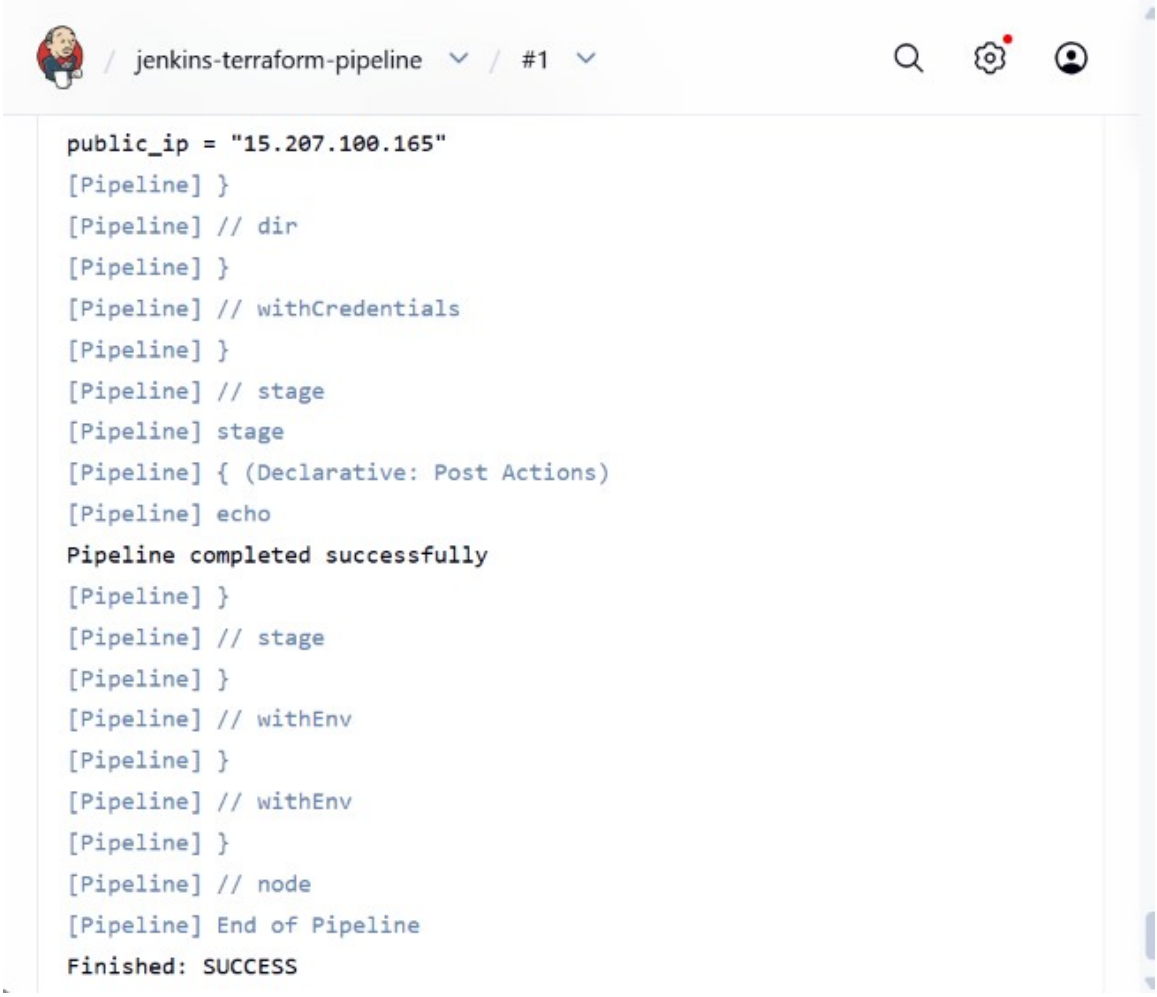


Figure 13: End of the console output, showing the `public_ip` output, `Pipeline completed successfully` and `Finished: SUCCESS`.

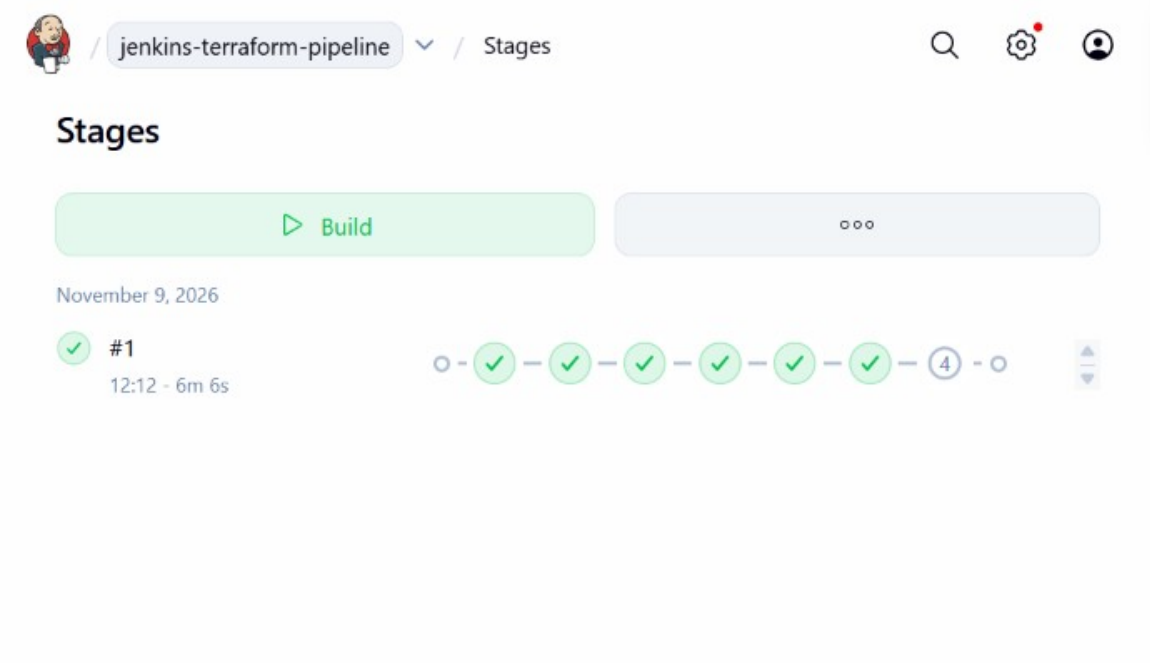


Figure 14: Stage view of build #1 (6 min 6 s).

6. Terraform configuration

The configuration in the `terraform/` folder defines one EC2 instance and the security group it uses, split across three files so it stays readable.

File	Contents
`main.tf`	`terraform` block (Terraform 1.3 or newer, AWS provider `~> 5.0`), the `aws` provider, the AMI lookup, the security group and the EC2 instance
`variables.tf`	`aws_region` (default `ap-south-1`) and `instance_type` (default `t3.micro`)
`outputs.tf`	`instance_id` and `public_ip` of the new instance

Resources in `main.tf`:

Block	Purpose
`data.aws_ami.amazon_linux`	Finds the latest Amazon Linux 2023 image, so no AMI ID is hard-coded
`aws_security_group.web_sg`	Named `jenkins-demo-sg`; allows inbound SSH (port 22) and HTTP (port 80) and all outbound traffic
`aws_instance.web`	The server: that AMI, the instance type from the variable, the security group, and the tag `Name = jenkins-terraform-demo`

Using variables for the region and instance type means the same files can be reused in another region or with a different size without editing the resources. For a real deployment the SSH rule would be limited to a trusted IP range instead of `0.0.0.0/0`.

7. Terraform validation and provisioning workflow

The configuration passed `terraform validate`, produced a correct plan of 2 resources to add, and was applied through Jenkins to create a running EC2 instance (Figures 15 and 16).

Step	Result
`terraform init`	Ran as the Terraform Init stage in Jenkins and passed
`terraform fmt -check`	No formatting problems reported
`terraform validate`	`Success! The configuration is valid.`
`terraform plan`	`Plan: 2 to add, 0 to change, 0 to destroy.` with the outputs `instance_id` and `public_ip` shown as known after apply
`terraform apply`	Run by the pipeline from the saved plan `tfplan` after approval

The plan reads the AMI (`ami-00531e78dd10fb3be`, Amazon Linux 2023) and will create two resources: `aws_security_group.web_sg` (`jenkins-demo-sg`, allowing ports 22 and 80) and `aws_instance.web` (type `t3.micro`, tag `Name = jenkins-terraform-demo`). The plan was run both on the command line and inside the Jenkins pipeline, with the same result.

Provisioning result, checked in the AWS Console (Figure 16):

Item	Value
Instance name	`jenkins-terraform-demo`
Instance ID	`i-00faadbbb73f5543f`
State	Running
Region	Asia Pacific (Mumbai), `ap-south-1`
Public IPv4 address	`15.207.100.165` (matches the `public_ip` output in the Jenkins log)
Private IPv4 address	`172.31.7.220`

```
Success! The configuration is valid.

data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 4s [id=ami-00531e78dd10fb3be]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
```

Figure 15: `terraform validate` succeeding and `terraform plan` reading the AMI and listing the resources to create.

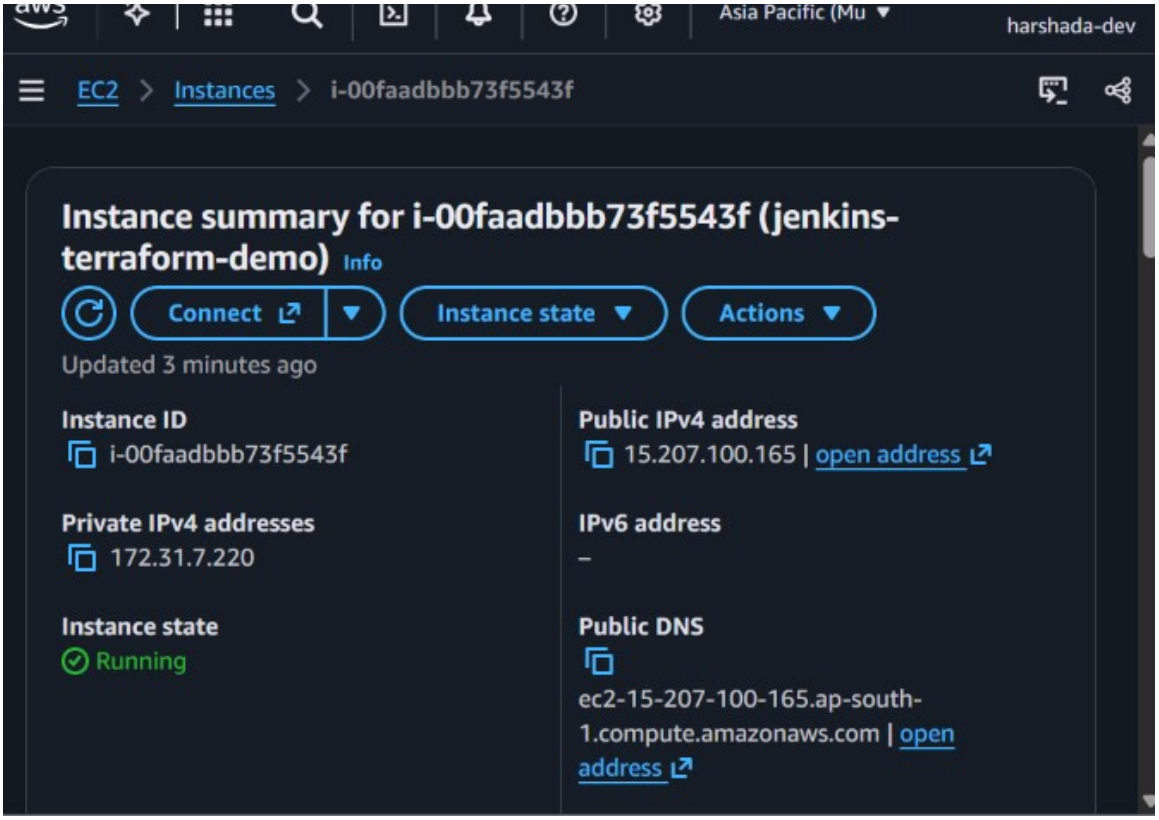


Figure 16: The `jenkins-terraform-demo` instance in the Running state in the AWS Console (Mumbai).

8. Integration of CI/CD and Infrastructure as Code

CI/CD automates how code is built, tested and delivered, and Infrastructure as Code automates how the servers it runs on are created. Jenkins joins the two in one pipeline, so a single change in GitHub can lead to tested code and a described, reviewable infrastructure change.

How they connect in this project:

1. A developer pushes code and the `Jenkinsfile` to GitHub.
2. Jenkins checks out the repository and runs the build and test stages. A failure here stops the pipeline before any infrastructure is touched.
3. Jenkins runs `terraform init`, `fmt` and `validate` to check that the infrastructure code is correct.
4. `terraform plan` shows exactly what would change in AWS and saves it as `tfplan`.
5. A person approves the plan.
6. `terraform apply` creates the security group and EC2 instance from that saved plan, so what was reviewed is exactly what gets built.

Why this is useful:

- **Repeatable:** the same files create the same infrastructure every time, with no manual console clicks.
- **Version controlled:** the infrastructure definition lives in Git next to the application, so every change has a history.
- **Safer:** tests, validation, a plan preview and a manual approval gate catch mistakes before resources are created.
- **Faster:** one button (or one push) runs the whole path from source code to a running server.
- **Secure:** the AWS key is held in the Jenkins credential store and exposed only to the stages that need it.

The next section lists everything submitted for this activity, including the recorded demonstration.

9. Conclusion and submission contents

A Jenkins pipeline now takes code from GitHub, builds and tests it, validates and plans Terraform, and after approval creates an AWS EC2 instance, with the whole run finishing in `Finished: SUCCESS`. The activity showed that application delivery and infrastructure provisioning can sit in one automated, repeatable and reviewable workflow.

Expected submission	Where to find it
Jenkinsfile	`Jenkinsfile` in the GitHub repository
Source-code repository	Harshadagujar2005/jenkins-terraform-demo
Terraform `.tf` files	`terraform/main.tf`, `variables.tf`, `outputs.tf` in the repository
Execution logs and screenshots	Figures 1 to 16 in this report, and `docs/screenshots/` and `logs/` in the repository
Validation output	Section 7 of this report and `logs/` in the repository
Recorded demonstration	Video submitted with this report